

assignment1

March 17, 2026

1 Engineering Modelling 2 - Assignment 1

Yaman Ashqar - a1884774

1.1 Question 1(a)

We have three data points at equally spaced nodes $x_0 = 0$, $x_1 = h$, $x_2 = 2h$, with values f_0 , f_1 , f_2 .

The Lagrange form of the interpolating polynomial is:

$$P_2(x) = f_0 L_0(x) + f_1 L_1(x) + f_2 L_2(x)$$

where the Lagrange basis polynomials are:

$$L_0(x) = \frac{(x-h)(x-2h)}{(0-h)(0-2h)} = \frac{(x-h)(x-2h)}{2h^2}$$

$$L_1(x) = \frac{(x-0)(x-2h)}{(h-0)(h-2h)} = \frac{x(x-2h)}{-h^2}$$

$$L_2(x) = \frac{(x-0)(x-h)}{(2h-0)(2h-h)} = \frac{x(x-h)}{2h^2}$$

the Lagrange quadratic polynomial is:

$$P_2(x) = \frac{(x-h)(x-2h)}{2h^2} f_0 - \frac{x(x-2h)}{h^2} f_1 + \frac{x(x-h)}{2h^2} f_2$$

1.2 Question 1(b)

substituting $x = \frac{h}{2}$ into the Lagrange polynomial $P_2(x)$ from part (a):

evaluate each basis polynomial at $x = h/2$:

$$L_0\left(\frac{h}{2}\right) = \frac{\left(\frac{h}{2} - h\right)\left(\frac{h}{2} - 2h\right)}{2h^2} = \frac{\left(-\frac{h}{2}\right)\left(-\frac{3h}{2}\right)}{2h^2} = \frac{\frac{3h^2}{4}}{2h^2} = \frac{3}{8}$$

$$L_1\left(\frac{h}{2}\right) = \frac{\frac{h}{2} \left(\frac{h}{2} - 2h\right)}{-h^2} = \frac{\frac{h}{2} \cdot \left(-\frac{3h}{2}\right)}{-h^2} = \frac{-\frac{3h^2}{4}}{-h^2} = \frac{3}{4}$$

$$L_2\left(\frac{h}{2}\right) = \frac{\frac{h}{2} \left(\frac{h}{2} - h\right)}{2h^2} = \frac{\frac{h}{2} \cdot \left(-\frac{h}{2}\right)}{2h^2} = \frac{-\frac{h^2}{4}}{2h^2} = -\frac{1}{8}$$

put it all together:

$$f\left(\frac{h}{2}\right) \approx P_2\left(\frac{h}{2}\right) = \frac{3}{8}f_0 + \frac{3}{4}f_1 - \frac{1}{8}f_2$$

$$f\left(\frac{h}{2}\right) \approx \frac{1}{8}(3f_0 + 6f_1 - f_2)$$

1.3 Question 1(c)

The polynomial interpolation error theorem states that for a polynomial $P_n(x)$ interpolating f at $n+1$ distinct nodes $x_0, x_1, \dots, x_n$, the error is:

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{j=0}^n (x - x_j)$$

for some ξ in the interval containing x and all the nodes.

Applying to our problem:

we have $n = 2$ (quadratic), nodes $x_0 = 0$, $x_1 = h$, $x_2 = 2h$, and we evaluate at $x = h/2$, so:

$$f\left(\frac{h}{2}\right) - P_2\left(\frac{h}{2}\right) = \frac{f^{(3)}(\xi)}{3!} \left(\frac{h}{2} - 0\right) \left(\frac{h}{2} - h\right) \left(\frac{h}{2} - 2h\right)$$

evaluate the node product:

$$\left(\frac{h}{2}\right) \left(\frac{h}{2} - h\right) \left(\frac{h}{2} - 2h\right) = \left(\frac{h}{2}\right) \left(-\frac{h}{2}\right) \left(-\frac{3h}{2}\right) = \frac{3h^3}{8}$$

therefore the error is:

$$f\left(\frac{h}{2}\right) - P_2\left(\frac{h}{2}\right) = \frac{f^{(3)}(\xi)}{6} \cdot \frac{3h^3}{8} = \frac{h^3}{16} f^{(3)}(\xi)$$

for some $\xi \in (0, 2h)$.

1.4 Question 1(d)

With $h = 0.5$, we have the data points:

$$f_0 = \sin(0) = 0, \quad f_1 = \sin(2 \cdot 0.5) = \sin(1), \quad f_2 = \sin(2 \cdot 1.0) = \sin(2)$$

```
[ ]: import numpy as np

h = 0.5

f0 = np.sin(2 * 0) # sin(0)
f1 = np.sin(2 * h) # sin(1)
f2 = np.sin(2 * 2*h) # sin(2)

print(f"f0 = sin(0) = {f0:.4f}")
print(f"f1 = sin(1) = {f1:.4f}")
print(f"f2 = sin(2) = {f2:.4f}")
```

```
f0 = sin(0) = 0.0000
f1 = sin(1) = 0.8415
f2 = sin(2) = 0.9093
```

1.4.1 Part (i)

From part (b):

$$f\left(\frac{h}{2}\right) \approx \frac{1}{8}(3f_0 + 6f_1 - f_2)$$

```
[2]: approx = (1/8) * (3*f0 + 6*f1 - f2)
print(f"Approximation P2(h/2) = (1/8)(3f0 + 6f1 - f2) = {approx:.4f}")
```

```
Approximation P2(h/2) = (1/8)(3f0 + 6f1 - f2) = 0.5174
```

1.4.2 Part (ii)

From part (c), the error is:

$$\left|f\left(\frac{h}{2}\right) - P_2\left(\frac{h}{2}\right)\right| = \frac{h^3}{16} |f^{(3)}(\xi)|, \quad \xi \in (0, 2h)$$

for $f(x) = \sin(2x)$, the derivatives cycle as:

$$f'(x) = 2\cos(2x), \quad f''(x) = -4\sin(2x), \quad f^{(3)}(x) = -8\cos(2x)$$

so $|f^{(3)}(\xi)| = 8|\cos(2\xi)| \leq 8$ for all ξ . The bound is maximised by taking $|f^{(3)}(\xi)| = 8$.

```
[3]: # third derivative of sin(2x) is -8cos(2x), so |f'''(xi)| <= 8
max_third_deriv = 8

error_bound = (h**3 / 16) * max_third_deriv
print(f"Error bound = (h^3 / 16) * 8 = ({h}^3 / 16) * 8 = {error_bound:.4f}")
```

```
Error bound = (h^3 / 16) * 8 = (0.5^3 / 16) * 8 = 0.0625
```

1.4.3 Part (iii)

the exact value is $f(h/2) = \sin(2 \cdot h/2) = \sin(h) = \sin(0.5)$.

```
[ ]: exact = np.sin(2 * (h/2)) # sin(h) = sin(0.5)
    actual_error = abs(exact - approx)

    print(f"Exact value sin(h) = sin(0.5) = {exact:.4f}")
    print(f"Approximation P2(h/2) = {approx:.4f}")
    print(f"Actual absolute error = {actual_error:.4f}")
    print(f"Theoretical error bound = {error_bound:.4f}")
    print(f"\nIs actual error <= bound? {actual_error <= error_bound}")
```

Exact value sin(h) = sin(0.5) = 0.4794

Approximation P2(h/2) = 0.5174

Actual absolute error = 0.0380

Theoretical error bound = 0.0625

Is actual error <= bound? True

the actual absolute error is less than the theoretical bound, as expected. The bound is not tight because we used the global maximum $|f^{(3)}(\xi)| = 8$ (i.e. $|\cos(2\xi)| = 1$), whereas the true $\xi \in (0, 1.0)$ gives a value of $|\cos(2\xi)| < 1$. The error theorem guarantees only the existence of such a ξ , it doesn't tell us its exact location, so we must take the worst case value over the whole interval, which overestimates the true error.

1.5 Question 2(a)

the composite trapezoidal rule for $\int_a^b g(r) dr$ over n subintervals with uniform spacing $h = (b-a)/n$ is:

$$\int_a^b g(r) dr \approx \frac{h}{2} \left[g(r_0) + 2 \sum_{j=1}^{n-1} g(r_j) + g(r_n) \right]$$

applying this to $I_z = 2\pi \int_0^R r^3 t(r) dr$, we set $g(r) = r^3 t(r)$, $a = 0$, $b = R$.

```
[6]: import numpy as np

def moment_inertia(thickness, rmax, n):
    """
    approximate Iz = 2*pi * integral_0^rmax r^3 * t(r) dr
    using the composite trapezoidal rule with n subintervals.

    parameters
    -----
    thickness : callable
        Function t(r) giving disk thickness at radius r.
    rmax : float
```

```

    Maximum radius R of the disk.
    n : int
        Number of subintervals.

    returns
    -----
    float
        Approximation of Iz.
    """
    h = rmax / n # uniform step size, a=0 so h=(rmax-0)/n
    r = np.linspace(0, rmax, n + 1) # n+1 equally spaced nodes r_0, ..., r_n

    g = r**3 * thickness(r) # integrated g(r) = r^3 * t(r) at all nodes

    # composite trapezoidal rule: h/2 * (g_0 + 2*g_1 + ... + 2*g_{n-1} + g_n)
    integral = (h / 2) * (g[0] + 2 * np.sum(g[1:-1]) + g[-1])

    return 2 * np.pi * integral

```

Verification: for a flat disk of constant thickness $t(r) = t_0$ and radius R , the exact value is:

$$I_z = 2\pi t_0 \int_0^R r^3 dr = 2\pi t_0 \cdot \frac{R^4}{4} = \frac{\pi t_0 R^4}{2}$$

```

[9]: # quick verification against known exact solution
    t0, R = 1.0, 1.0
    exact_flat = np.pi * t0 * R**4 / 2

    approx_flat = moment_inertia(thickness=lambda r: t0, rmax=R, n=1000)

    print(f"Exact Iz (flat disk): {exact_flat:.6f}")
    print(f"Approx Iz (n=1000): {approx_flat:.6f}")
    print(f"Absolute error: {abs(exact_flat - approx_flat):.2e}")

```

Exact Iz (flat disk): 1.570796

Approx Iz (n=1000): 1.570798

Absolute error: 1.57e-06

1.6 Question 2(b)

for this thickness function, the integral becomes:

$$g(r) = r^3 t(r) = r^3 \cdot \frac{r}{2\pi} = \frac{r^4}{2\pi}$$

so we need to approximate:

$$I_z = 2\pi \int_0^1 \frac{r^4}{2\pi} dr = \int_0^1 r^4 dr$$

1.6.1 Part (i)

with $R = 1$, $n = 4$: step size $h = \frac{1-0}{4} = 0.25$

the nodes are $r_j = jh$ for $j = 0, 1, 2, 3, 4$:

j	r_j	$g(r_j) = r_j^4/2\pi$
0	0.00	$0.0000/2\pi = 0.0000$
1	0.25	$0.003906/2\pi = 0.000622$
2	0.50	$0.062500/2\pi = 0.009947$
3	0.75	$0.316406/2\pi = 0.050338$
4	1.00	$1.000000/2\pi = 0.159155$

applying the composite trapezoidal rule (eq. 2.2):

$$\begin{aligned}
 I_{\text{trap}} &= \frac{h}{2} [g(r_0) + g(r_4) + 2(g(r_1) + g(r_2) + g(r_3))] \\
 &= \frac{0.25}{2} [0.0000 + 0.159155 + 2(0.000622 + 0.009947 + 0.050338)] \\
 &= 0.125 \times [0.159155 + 2 \times 0.060907] \\
 &= 0.125 \times 0.280969 = 0.035121
 \end{aligned}$$

then:

$$I_z = 2\pi \times I_{\text{trap}} = 2\pi \times 0.035121 \approx 0.2207$$

```
[8]: import numpy as np

def moment_inertia(thickness, rmax, n):
    h = rmax / n
    r = np.linspace(0, rmax, n + 1)
    g = r**3 * thickness(r)
    integral = (h / 2) * (g[0] + 2 * np.sum(g[1:-1]) + g[-1])
    return 2 * np.pi * integral

# test case: t(r) = r / (2*pi), R = 1, n = 4
thickness = lambda r: r / (2 * np.pi)
result = moment_inertia(thickness, rmax=1, n=4)
```

```

print(f"moment_inertia result (n=4): {result:.4f}")

# exact value for reference:  $I_z = \int_0^1 r^4 dr = 1/5 = 0.2000$ 
exact = 1/5
print(f"Exact value: {exact:.4f}")
print(f"Absolute error: {abs(result - exact):.4f}")

```

```

moment_inertia result (n=4): 0.2207
Exact value: 0.2000
Absolute error: 0.0207

```

1.6.2 Part (ii)

The function returns the same value as the by-hand calculation, confirming the implementation is correct. The small remaining error relative to the exact value $I_z = \int_0^1 r^4 dr = \frac{1}{5} = 0.2000$ is expected, the trapezoidal rule with only $n = 4$ subintervals is $\mathcal{O}(h^2)$ accurate, so we expect the approximation to improve as n increases.

1.7 Question 2(c)

$$t(r) = e^{-0.05r^2} + 1 + \tanh(4r - 16), \quad 0 \leq r \leq 5$$

```

[10]: import numpy as np

def moment_inertia(thickness, rmax, n):
    h = rmax / n
    r = np.linspace(0, rmax, n + 1)
    g = r**3 * thickness(r)
    integral = (h / 2) * (g[0] + 2 * np.sum(g[1:-1]) + g[-1])
    return 2 * np.pi * integral

thickness_flywheel = lambda r: np.exp(-0.05 * r**2) + 1 + np.tanh(4*r - 16)

# experiment with increasing n to check convergence to 3 significant figures
ns = [10, 50, 100, 500, 1000, 5000]
print(f"{'n':>6} {'Iz':>12}")
print("-" * 22)
for n in ns:
    Iz = moment_inertia(thickness_flywheel, rmax=5, n=n)
    print(f"{'n':>6} {'Iz':>12.6f}")

```

n	Iz
10	1607.825562
50	1591.151931
100	1590.546415
500	1590.352642

1000	1590.346586
5000	1590.344649

1.7.1 Output

n	I _z

10	532.073553
50	530.809612
100	530.778600
500	530.771340
1000	530.771158
5000	530.771121

the value stabilises at $I_z \approx 531$ to three significant figures from $n = 100$ onwards, the first three significant figures (5, 3, 0) are unchanged between $n = 100$ and $n = 5000$. we therefore take $n = 100$ as sufficient, giving:

$$I_z \approx 531$$

1.8 Question 3

we are given the finite difference formula:

$$f''(x) \approx \frac{f(x-2h) - f(x-h) - f(x+h) + f(x+2h)}{ah^2}$$

1.9 Question 3(a)

Expand each term using Taylor's theorem about x :

$$f(x+h) = f(x) + f'(x)h + \frac{f''(x)}{2}h^2 + \frac{f'''(x)}{6}h^3 + \frac{f^{(4)}(x)}{24}h^4 + \dots$$

$$f(x-h) = f(x) - f'(x)h + \frac{f''(x)}{2}h^2 - \frac{f'''(x)}{6}h^3 + \frac{f^{(4)}(x)}{24}h^4 + \dots$$

$$f(x+2h) = f(x) + 2f'(x)h + 2f''(x)h^2 + \frac{4}{3}f'''(x)h^3 + \frac{2}{3}f^{(4)}(x)h^4 + \dots$$

$$f(x-2h) = f(x) - 2f'(x)h + 2f''(x)h^2 - \frac{4}{3}f'''(x)h^3 + \frac{2}{3}f^{(4)}(x)h^4 + \dots$$

1.10 Question 3(b)

from the Taylor expansions in part (a), the numerator evaluates to:

$$f(x-2h) - f(x-h) - f(x+h) + f(x+2h) = 3f''(x)h^2 + \mathcal{O}(h^4)$$

substituting into (3):

$$\frac{f(x-2h) - f(x-h) - f(x+h) + f(x+2h)}{ah^2} = \frac{3f''(x)h^2 + \mathcal{O}(h^4)}{ah^2} = \frac{3}{a}f''(x) + \mathcal{O}(h^2)$$

for this to approximate $f''(x)$, we require $\frac{3}{a} = 1$, hence:

$$a = 3$$

the finite difference formula is therefore:

$$f''(x) \approx \frac{f(x-2h) - f(x-h) - f(x+h) + f(x+2h)}{3h^2}$$

with error $\mathcal{O}(h^2)$, i.e. the formula is **second-order accurate**.

1.11 Question 3(c)

from part (b), the formula with $a = 3$ is:

$$f''(x) \approx \frac{f(x-2h) - f(x-h) - f(x+h) + f(x+2h)}{3h^2}$$

```
[14]: import numpy as np

f = lambda x: np.cos(x**2)

x, h = 1.0, 0.1

fd2_approx = (f(x-2*h) - f(x-h) - f(x+h) + f(x+2*h)) / (3 * h**2)

# exact second derivative: f'(x) = -2x*sin(x^2), f''(x) = -2*sin(x^2) - 4x^2*cos(x^2)
fd2_exact = -2*np.sin(x**2) - 4*x**2*np.cos(x**2)

print(f"Finite difference approximation of f''(1): {fd2_approx:.6f}")
print(f"Exact value of f''(1): {fd2_exact:.6f}")
print(f"Absolute error: {abs(fd2_approx - fd2_exact):.2e}")
```

```
Finite difference approximation of f''(1): -3.666612
Exact value of f''(1): -3.844151
Absolute error: 1.78e-01
```

1.11.1 Output

```
Finite difference approximation of f''(1): -3.666612
Exact value of f''(1): -3.844151
Absolute error: 1.78e-01
```

The finite difference approximation gives $f''(1) \approx -3.6666$. The absolute error of 1.78×10^{-1} is relatively large for $h = 0.1$, but this is consistent with a second-order method whose $\mathcal{O}(h^2)$ constant is large — the fourth derivative of $\cos(x^2)$ is significant at $x = 1$, inflating the prefactor on the error term.

To confirm the formula is still second-order accurate despite the large error constant, we check that the error quarters each time h halves:

```
[15]: for h_test in [0.1, 0.05, 0.025]:
        approx = (f(x-2*h_test) - f(x-h_test) - f(x+h_test) + f(x+2*h_test)) / 12
        ↪ (3*h_test**2)
        print(f"h = {h_test:.3f}    error = {abs(approx - fd2_exact):.4e}")
```

```
h = 0.100    error = 1.7754e-01
h = 0.050    error = 4.4343e-02
h = 0.025    error = 1.1082e-02
```

1.11.2 Output

```
h = 0.100    error = 1.7754e-01
h = 0.050    error = 4.4343e-02
h = 0.025    error = 1.1082e-02
```

The error reduces by a factor of approximately 4 each time h is halved, confirming $\mathcal{O}(h^2)$ convergence as expected from part (b).